NSBE NATIONAL SOCIETY OF BLACK ENGINEERS

Building and Deploying Retrieval-Augmented Generation (RAG) Systems for Quality Business Impact

Harnessing Generative AI and Retrieval Models for Business Innovation

Ridwan Amure

March 7th 2025

1975 to 2025 YEARS

INSPIRE! EXCEL! IMPACT! INSPIRE! EXCEL! IMPACT! INSPIRE! EXCEL! IMPACT!

How can businesses generate accurate, context-aware responses efficiently while leveraging both retrieval and generative AI?

"The future belongs to those who learn more skills and combine them in creative ways." - Robert Greene

Hi there, my name is

RIDWAN AMURE

- Machine Learning Engineer
- Researcher at COSMOS Research Center

AGENDA

1

Understand

Understand the core principles of RAG systems.

2

Learn

Learn to build a simple RAG system from scratch.

3

Explore

Explore real-world business applications of RAG systems.

4

Gain

Gain hands-on experience in developing RAG systems.

WHAT IS

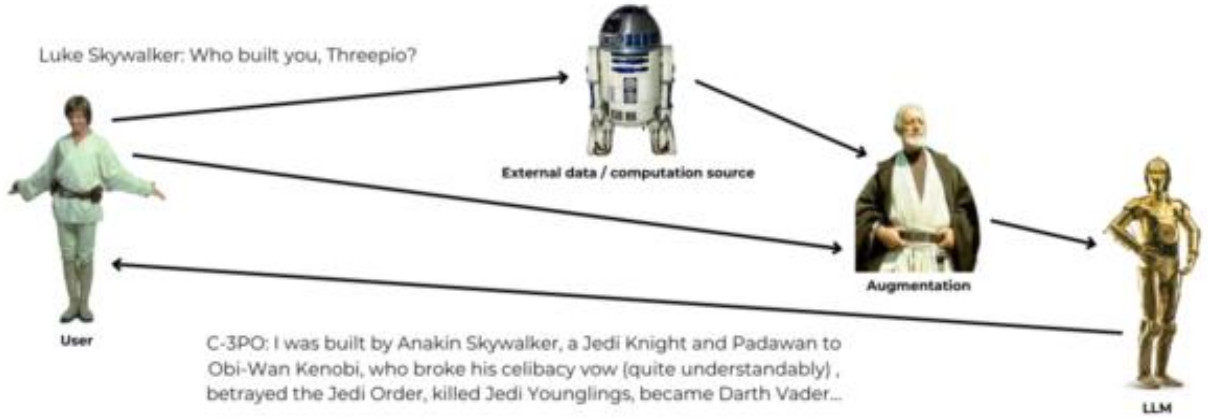
RAG

RAG = Information retrieval + Generative AI

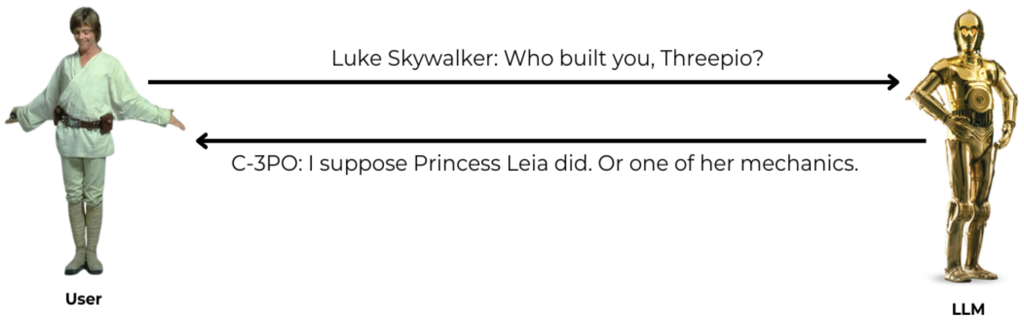
RAG = Contextual Information + Generative AI

RAG

LLMs with RAG



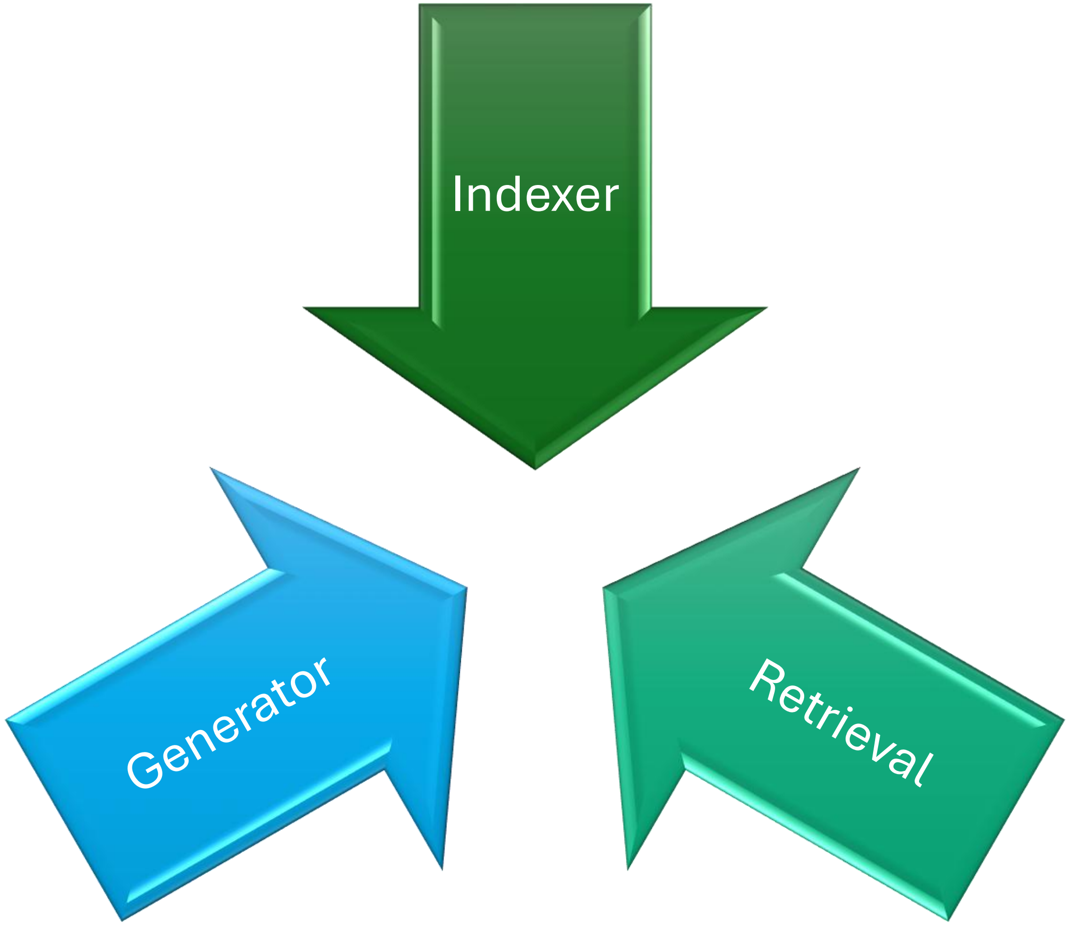
LLMs without RAG

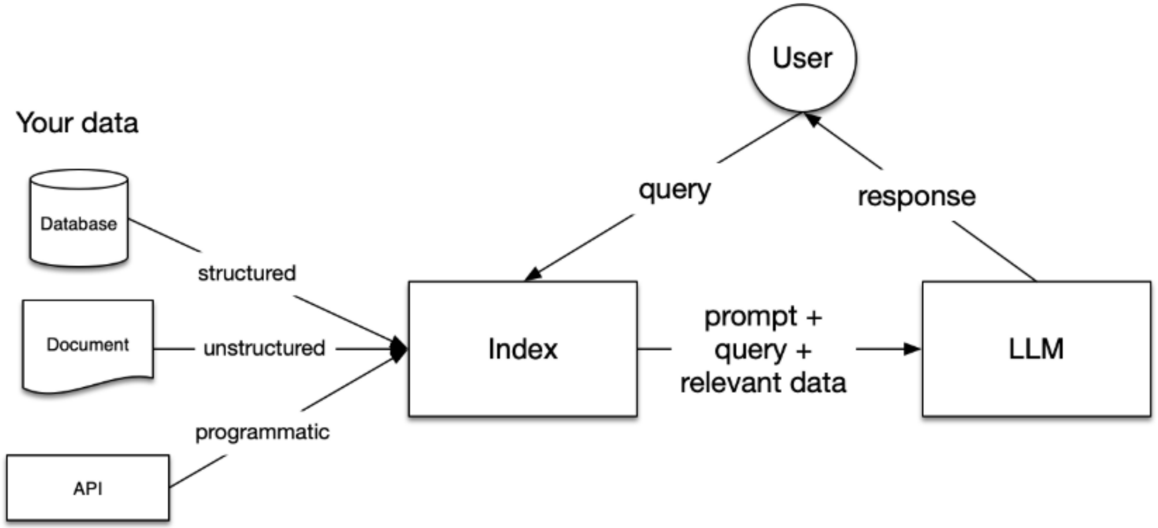


https://www.globalnerdy.com/2024/04/16/retrieval-augmented-generation-explained-star-wars-style/

CORE COMPONENTS OF

RAG





https://docs.llamaindex.ai/en/stable/understanding/rag/

STAGES WITHIN

RAG

LOADING DATA

- Usually consists of three stages
  - Load the data
  - Transform the data
  - Index and store the data
- Depends on data format and source
- Depends on business needs and requirements
- Transformation:
  - Conversion of data into useable format
  - Include chunking, extracting metadata, and embedding each chunk.
  - Directly impacts indexing and retrieval

INDEXING

- **Organizing and storing documents or data in a structured format**
- **Involves creating a searchable representation of knowledge sources**
- **Involves vectorization of chunks using techniques like**
  - TF-IDF, BM25, Dense Passage Retrieval (DPR), or embeddings from transformer-based models (e.g., BERT, FAISS).
- **End result is a document(chunk) and its vector representation**

INDEXING

Vector Store	Knowledge Graph	Summary Index
1 Split document into Nodes	1 Leverage inherent word relationships in documents	1 Best suited for summary task
2 Create vector embeddings of the text of every node	2 Uses KG to represent entity relationship and create vector representation	2 Stores all of the Documents and returns all of them to the query engine.
3 Makes the embedding available for querying by LLM	3 Best use if data is a set of interconnected concepts	

STORING

- Storing prevents recurring time and cost of re-indexing
- Index can be stored
  - Locally
  - ChromaDB
  - AstraDB
  - Databricks
  - MongoDB
  - DuckDB
  - FAISS

QUERYING

- Put simply, Querying is a prompt call to an LLM
- Involves using a querying engine
- Stages:
  - Retrieval is when you find and return the most relevant documents for your query from your Index.
  - Postprocessing is when the Nodes retrieved are optionally reranked, transformed, or filtered.
  - Response synthesis is when your query, your most-relevant data and your prompt are combined and sent to your LLM to return a response.

USE CASES IN BUSINESS

Enterprise Knowledge Management

Problem: Employees waste significant time searching for internal documentation, policies, and best practices.

Solution: A **RAG-powered enterprise search assistant** can retrieve relevant policies, manuals, and best practices from large internal knowledge bases.

Automated Customer Support & Chatbots

Problem: Traditional chatbots rely on pre-written responses, leading to inaccurate or irrelevant replies.

Solution: RAG enables context-aware, real-time AI chatbots that pull relevant data from FAQs, support tickets, and product documentation.

Financial Research & Market Analysis

Problem: Analysts spend too much time sifting through reports, earnings calls, and news articles.

Solution: A RAG-based system can **retrieve and summarize** relevant financial insights, reducing research time.

SECTION BREAK / LIVE DEMO

TECHNIQUES FOR

BUILDING PRODUCTION -GRADE RAG

STEP ONE

•Decoupling chunks used for retrieval vs. chunks used for synthesis

STEP TWO

•Structured Retrieval for Larger Document Sets

STEP THREE

•Dynamically Retrieve Chunks Depending on your Task

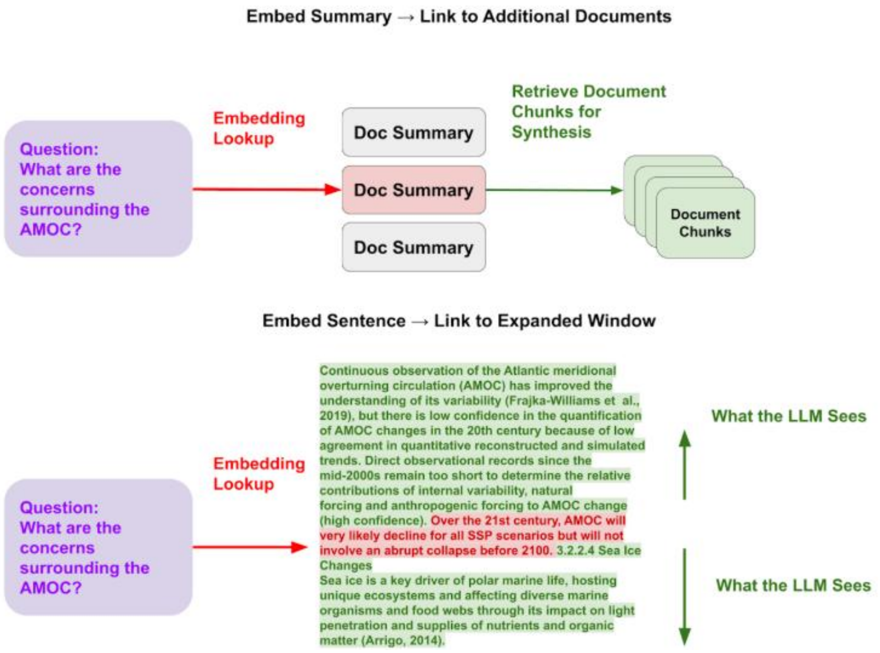
STEP FOUR

Optimize context embeddings

DECOUPLING CHUNKS

USED FOR RETRIEVAL

- The optimal chunk representation for retrieval might be different than the optimal consideration used for synthesis
- Two main techniques
  - Embed a document summary, which links to chunks associated with the document.
  - Embed a sentence, which then links to a window around the sentence.

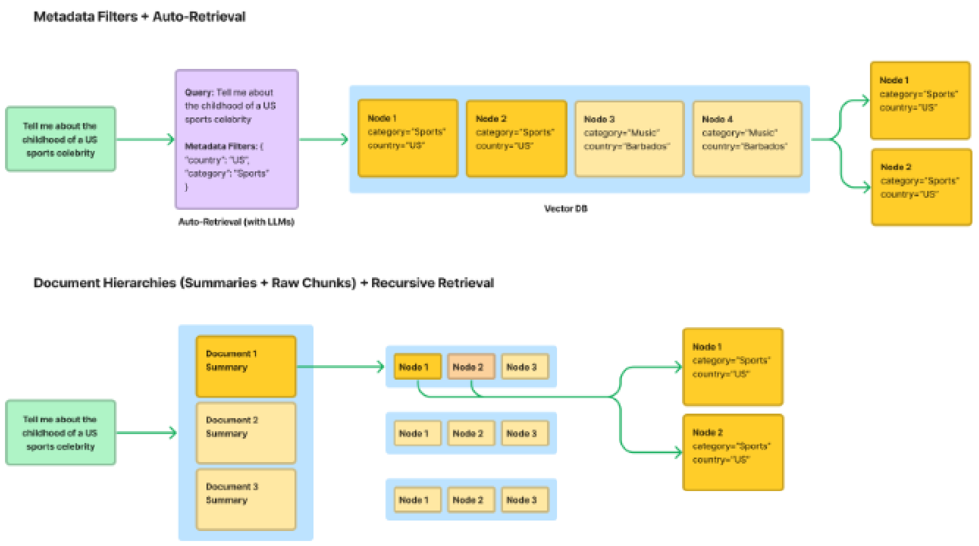


https://docs.llamaindex.ai/en/stable/optimizing/production_rag/

STRUCTURED RETRIEVAL FOR LARGER

DOCUMENT SETS

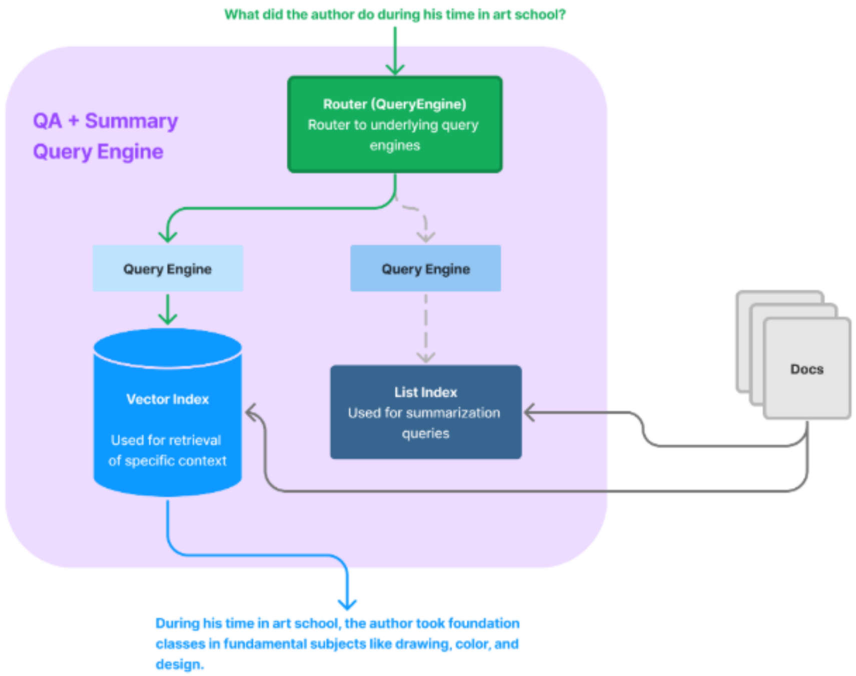
- Standard RAG stack (top-k retrieval + basic text splitting) doesn't do well as the number of documents scales up
- Structured information might help with more precise retrieval
- Key techniques
  - Metadata Filters + Auto Retrieval
  - Store Document Hierarchies (summaries -> raw chunks) + Recursive Retrieval



https://docs.llamaindex.ai/en/stable/optimizing/production_rag/

DYNAMICALLY RETRIEVE CHUNKS

- RAG are not only used for question-answering
- Queries can involve summarization, hence different use cases may require different retrieval techniques.
- Key techniques
  - Combine retrieval engines
  - Use Agents

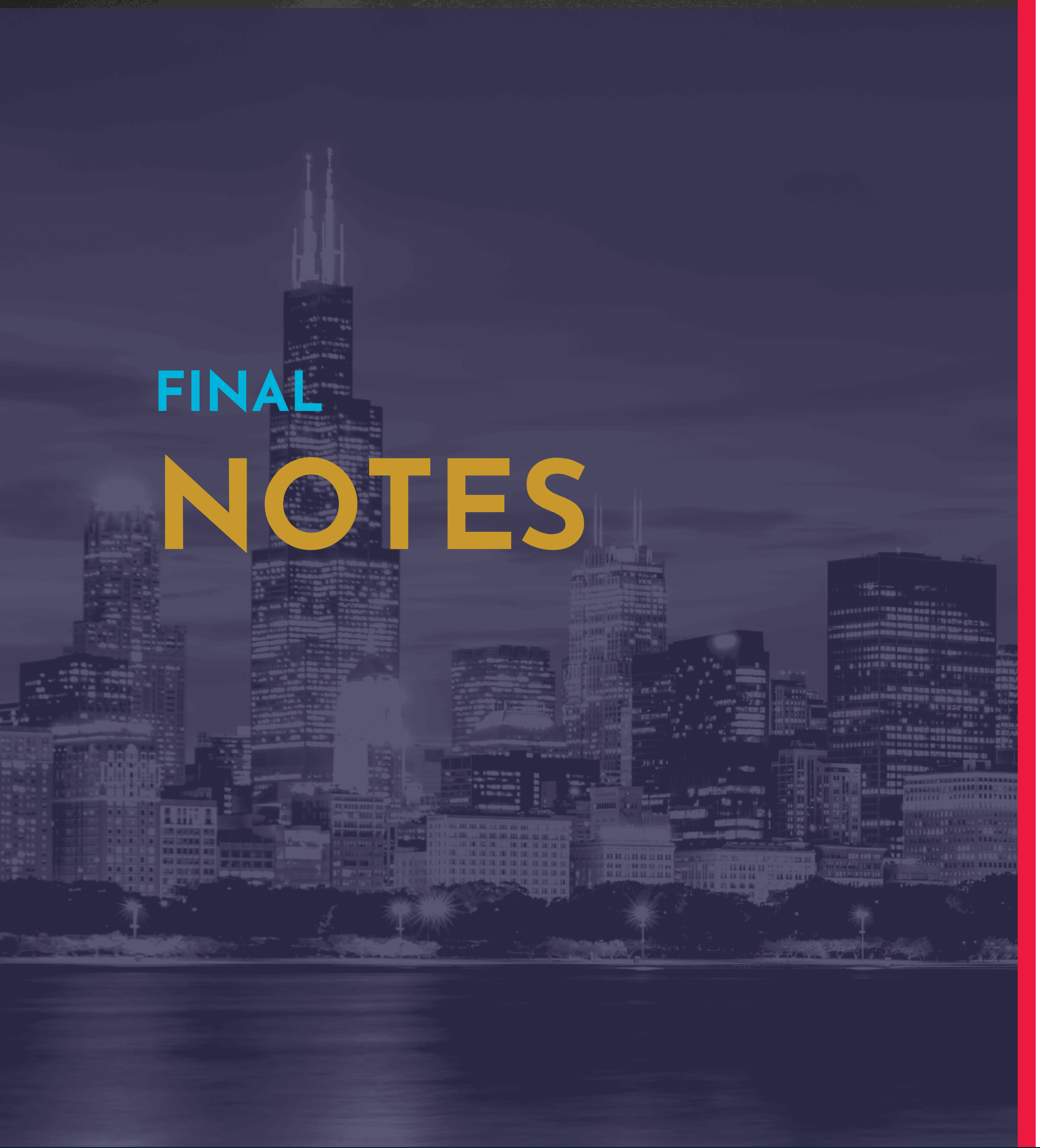


https://docs.llamaindex.ai/en/stable/optimizing/production_rag/

OPTIMIZE
CONTEXT EMBEDDINGS

- Optimizations of embedding for better retrieval
- Pre-trained models may not capture the salient properties of the data relevant to your use case.
- Key techniques
  - Fine tuning embedding models

FINAL NOTES



- RAG Bridge Retrieval and Generation
- Business Efficiency & Automation
- Scalability & Real-Time Insights
- Deployability & Integration

THANK YOU